Lecture-12 [04 September 2026]

Modified Policy Iteration

Recap: Policy Iteration algorithm

- Initialisation: $\phi : \mathcal{S} \to \mathcal{A}$ deterministic decision rule

  $\Gamma \leftarrow \mathcal{S}$.

- Iterate: While Γ is non-empty:

  - Policy evaluation

    $\underline{v}' \leftarrow (I - \lambda P_\phi)^{-1} r_\phi$

    $\underline{v}'$ ← value function of the policy $\pi = \{\phi, \phi, \dots\} \in \Pi^{SD}$

  - Policy improvement

    Set $\phi' : \mathcal{S} \to \mathcal{A}$ as

    $$\phi'(s) \in \arg\max_{a \in \mathcal{A}} \left[r(s,a) + \lambda \sum_{s' \in \mathcal{S}} P(s'|s,a)\, v'(s') \right], \quad s \in \mathcal{S}.$$

    Choose $\phi'(s) = \phi(s)$ if $\phi(s)$ is an element of arg max.

  - $\Gamma \leftarrow \{ s \in \mathcal{S} : \phi'(s) \neq \phi(s) \}$.

  - $\phi \leftarrow \phi'$

- Upon termination, return ϕ and $\underline{v}'$

$$P_\phi(s'|s) = \sum_a \phi(a|s)\, P(s'|s,a) = P(s'|s,\phi(s))$$

$$r_\phi(s) = \sum_a \phi(a|s)\, r(s,a) = r(s,\phi(s))$$

As established in the previous lecture, any two successive value iterates $\underline{v}, \underline{v}'$ coming from the policy iteration algorithm (i.e., $\phi \to \underline{v} = v^{\phi}_{\infty,\lambda} \to \phi' \to \underline{v}' = v^{\phi'}_{\infty,\lambda}$) satisfy

$$\underline{v}' = \underline{v} + (I - \lambda P_\phi)^{-1} B\underline{v} = \underline{v} + \sum_{n=0}^{\infty} \lambda^n P_\phi^n B\underline{v}.$$

Computing the inverse $(I - \lambda P_\phi)^{-1}$ is an expensive step ($O(|\mathcal{S}|^3)$ operations).

Instead of computing the inverse (an infinite sum), we can consider truncating the upper limit of the sum to $m \geq 0$. This leads to modified policy iteration.

Proposition 5.11

Fix $m \geq 0$ and $\underline{v} \in \mathbb{R}^{|\mathcal{S}|}$, and let $\underline{v} \to \phi' = \phi_{\underline{v}} \to \underline{v}' = v^{\phi'}_{\infty,\lambda}$

$$\underline{v}' = \underline{v} + \sum_{n=0}^{m} \left(\lambda P_{\phi_{\underline{v}}}\right)^n B\underline{v},$$

where $\phi_{\underline{v}}$ is the greedy decision rule for $\underline{v}$ given by

$$\phi_{\underline{v}}(s) = \arg\max_{a \in \mathcal{A}} \left[r(s,a) + \lambda \sum_{s' \in \mathcal{S}} P(s'|s,a)\, v(s') \right],$$

and $B\underline{v} = L\underline{v} - \underline{v}$. Then,

$$\underline{v}' = L^m_{\phi_{\underline{v}}} L\underline{v} = L^{m+1}_{\phi_{\underline{v}}} \underline{v}.$$

Remark: Truncating the upper limit of the sum to m is equivalent to applying $L_{\phi_{\underline{v}}}$ $m+1$ times.

Proof of Proposition 5.11

First, observe that $L\underline{v} = L_{\phi_{\underline{v}}} \underline{v}$. We then have

$$\begin{aligned}
\underline{v} + \sum_{n=0}^{m} \left(\lambda P_{\phi_{\underline{v}}}\right)^n B\underline{v} &= \underline{v} + \left(I + \lambda P_{\phi_{\underline{v}}} + \lambda^2 P^2_{\phi_{\underline{v}}} + \cdots + \lambda^m P^m_{\phi_{\underline{v}}}\right) B\underline{v} \\
&= \underline{v} + \left(I + \lambda P_{\phi_{\underline{v}}} + \lambda^2 P^2_{\phi_{\underline{v}}} + \cdots + \lambda^m P^m_{\phi_{\underline{v}}}\right)\left(L\underline{v} - \underline{v}\right) \\
&= \underline{v} + \left(I + \lambda P_{\phi_{\underline{v}}} + \lambda^2 P^2_{\phi_{\underline{v}}} + \cdots + \lambda^m P^m_{\phi_{\underline{v}}}\right)\left(L_{\phi_{\underline{v}}} \underline{v} - \underline{v}\right) \\
&= \underline{v} + \left(I + \lambda P_{\phi_{\underline{v}}} + \lambda^2 P^2_{\phi_{\underline{v}}} + \cdots + \lambda^m P^m_{\phi_{\underline{v}}}\right)\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} \underline{v} - \underline{v}\right) \\
&= r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} \underline{v} + \lambda P_{\phi_{\underline{v}}}\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} \underline{v} - \underline{v}\right) \\
&\quad + \lambda^2 P^2_{\phi_{\underline{v}}}\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} \underline{v} - \underline{v}\right) + \cdots \\
&\quad + \lambda^m P^m_{\phi_{\underline{v}}}\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} \underline{v} - \underline{v}\right) \\
&= r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} r_{\phi_{\underline{v}}} + \cdots + \lambda^m P^m_{\phi_{\underline{v}}} r_{\phi_{\underline{v}}} + \lambda^{m+1} P^{m+1}_{\phi_{\underline{v}}} \underline{v} \\
&= r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}}\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}}\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}}\left(\cdots\left(r_{\phi_{\underline{v}}} + \lambda P_{\phi_{\underline{v}}} \underline{v}\right)\right)\right)\right) \\
&= L^m_{\phi_{\underline{v}}} L\underline{v} = L^{m+1}_{\phi_{\underline{v}}} \underline{v}.
\end{aligned}$$

Modified Policy Iteration (MPI)

- **Initialise:**
  - Decision rule $\phi' : \mathcal{S} \to \mathcal{A}$
  - Specify $\varepsilon > 0$ and $\sigma > \left(\frac{1-\lambda}{\lambda}\right)\varepsilon$.
  - Specify $\underline{v} \in \mathbb{R}^{|\mathcal{S}|}$.
  - Specify a sequence of non-negative integers $\{m_1, m_2, \dots\}$.
  - $n \leftarrow 1$

- **Iterate:** While $\sigma \geq \left(\frac{1-\lambda}{\lambda}\right)\varepsilon$:
  - Update $\phi \leftarrow \phi'$
  - Evaluate:
    $$\underline{u} \leftarrow L^{m_n} \underline{v}$$
  - Improve:

    Set $\underline{v} \leftarrow L\underline{u}$,

    $$\phi'(s) = \arg\max_{a \in \mathcal{A}} \left[r(s,a) + \sum_{s' \in \mathcal{S}} P(s'|s,a)\, u(s') \right]$$
  - $\sigma \leftarrow \mathrm{sp}\left(\underline{v} - \underline{u}\right)$
  - $n \leftarrow n+1$.

- **Terminate:** Return $\phi_\varepsilon = \phi'$ and

  $$V_{\infty,\lambda,\varepsilon} = \underline{v} + \left(\frac{\lambda}{1-\lambda}\right) \min\left(\underline{v} - \underline{u}\right) \underline{1}.$$

Remarks:

(1) MPI reduces to value iteration when $m_n = 0 \quad \forall n$.
MPI reduces to policy iteration when $m_n = \infty \quad \forall n$.

(2) A typical choice for the initial $\underline{v}$ is

$$\underline{v} = \underline{v}_0 = \frac{r_{\min}}{1-\lambda}\,\underline{1}, \qquad r_{\min} = \min_{(s,a)} r(s,a).$$

(3) If $\phi_1, \phi_2, \dots$ denote the sequence of ϕ appearing in MPI, then

the value at iteration n (starting with $\underline{v}_0$) can be expressed as

$$L_{\phi_n}^{m_{n+1}} \cdots L_{\phi_2}^{m_2+1} L_{\phi_1}^{m_1} \underline{v}_0 .$$

(4) The choice of $\{m_1, m_2, \ldots\}$ affects the convergence rate.

Convergence Analysis for MPI

Given $m \geq 0$, let $W^m : \mathbb{R}^{|S|} \to \mathbb{R}^{|S|}$ be defined as

$$W^m \underline{v} = \underline{v} + \sum_{n=0}^{m} \left(\lambda P_{\phi_{\underline{v}}}\right)^n B\underline{v} \quad , \quad \underline{v} \in \mathbb{R}^{|S|}.$$

Then, the iterates of MPI can be represented as $\underline{v}_0, \underline{v}_1, \underline{v}_2, \ldots$ where

$$\underline{v}_n = W^{m_n} \underline{v}_{n-1} \quad \forall\ n \in \mathbb{N}.$$

Lemma 5.7

Suppose that $B\underline{v} \geq \underline{0}$ is satisfied for some $\underline{v} \in \mathbb{R}^{|S|}$. Then, for any $m \geq 0$:

(1) $v_{\infty,\lambda}^* \geq W^m \underline{v}$.

(2) $W^m \underline{v} \geq L\underline{v}$.

(3) $W^m \underline{v} \geq \underline{v}$.

(4) $B\left(W^m \underline{v}\right) \geq 0$.

Proof is along similar lines as that of Lemma 5.6 (see lecture 11) and is left as exercise.

Theorem 5.20 :

Let $\{m_1, m_2, \ldots\}$ be any sequence of non-negative integers, and let $\{\underline{v}_0, \underline{v}_1, \ldots\}$ denote the sequence of "$\underline{v}$" vectors that arise in MPI.

If $B\underline{v}_0 \geq \underline{0}$, then

$$\underline{v}_{n+1} \geq \underline{v}_n \quad \forall\ n, \qquad \| \underline{v}_n - v_{\infty,\lambda}^* \|_\infty \to 0 \quad \text{as } n \to \infty.$$

Proof of Theorem 5.20

Let $\underline{u}_0 = \underline{v}_0$. For each $n \in \mathbb{N}$, let

$$\underline{u}_{n+1} = L\underline{u}_n, \qquad \underline{v}_{n+1} = W_{m_{n+1}}\underline{v}_n.$$

Here, $\{\underline{u}_0, \underline{u}_1, \dots\}$ is the sequence of iterates of value iteration
$\{\underline{v}_0, \underline{v}_1, \dots\}$ is the sequence of iterates of MPI
} both start at $\underline{v}_0$

Claim:

$$\underline{u}_n \le \underline{v}_n \le \underline{v}^*_{\infty,\lambda} \quad \forall\, n \ge 0. \qquad \rightarrow (1)$$

Base case $n=0$ follows by noting that

$$B\underline{v}_0 \ge 0 \iff L\underline{v}_0 \ge \underline{v}_0$$
$$\Rightarrow L^2\underline{v}_0 \ge L\underline{v}_0 \ge \underline{v}_0$$
$$\Rightarrow L^n\underline{v}_0 \ge \underline{v}_0$$
$$\Rightarrow \lim_{n\to\infty} L^n\underline{v}_0 \ge \underline{v}_0$$
$$\Rightarrow \underline{v}^*_{\infty,\lambda} \ge \underline{v}_0 = \underline{u}_0.$$

Induction step: Assume $\underline{u}_k \le \underline{v}_k \le \underline{v}^*_{\infty,\lambda}$ and $B\underline{v}_k \ge \underline{0}$ $\forall\, k \in \{0, \dots, n\}$.

Then:

$$\underline{v}_{n+1} = W_{m_{n+1}}\underline{v}_n$$
$$\ge L\underline{v}_n \quad \text{(applying property (2) of Lemma 5.7, noting that } B\underline{v}_n \ge \underline{0} \text{ by induction hypothesis)}$$
$$\ge L\underline{u}_n \quad (\underline{v}_n \ge \underline{u}_n \Rightarrow L\underline{v}_n \ge L\underline{u}_n)$$
$$= \underline{u}_{n+1}.$$

Furthermore, $B\underline{v}_{n+1} \ge 0$ because $B(W_{m_{n+1}}\underline{v}_n) \ge 0$ as $B\underline{v}_n \ge \underline{0}$. (property (4) of Lemma 5.7)

$$\underline{v}_{n+1} = W_{m_{n+1}}\underline{v}_n$$
$$\ge \underline{v}_n \quad \text{(using property (3) of Lemma 5.7, noting that } B\underline{v}_n \ge \underline{0} \text{ by induction hypothesis)}.$$

Also, $\underline{v}_{n+1} \le \underline{v}^*_{\infty,\lambda}$ follows from property (1) of Lemma 5.7 (using $B\underline{v}_n \ge \underline{0}$).

This proves that $\{\underline{v}_n\}_{n\ge 0}$ is monotone & $\underline{u}_n \le \underline{v}_n \le \underline{v}^*_{\infty,\lambda}$ $\forall\, n \ge 0$.

Because $\underline{v}_0$ satisfies $L\underline{v}_0 \geq \underline{v}_0$, $\{\underline{u}_n\}_{n\geq 0}$ is monotone and converges to $v^*_{\infty,\lambda}$. From this, convergence of $\{\underline{v}_n\}$ to $v^*_{\infty,\lambda}$ follows.

Value Function Approximation
(Approximate Dynamic Programming)

- Exact evaluation of $v^*_{\infty,\lambda}$ is difficult in MDPs with large state/action spaces
  - $v^*_{\infty,\lambda}$ → element of $\mathbb{R}^{|\mathcal{S}|}$
- As a workaround, $v^*_{\infty,\lambda}$ is approximated by a low-dimensional parametric function
- Using the approximated value function, actions are chosen greedily:

$$\hat{\phi}^*(s) = \arg\max_{a\in\mathcal{A}} \left[r(s,a) + \sum_{s'\in\mathcal{S}} P(s'|s,a)\, \hat{v}^*_{\infty,\lambda}(s') \right]$$

  - $\hat{v}^*_{\infty,\lambda}$ → approximation of $v^*_{\infty,\lambda}$
- Some natural questions to consider:
  - How to obtain good approximations for $v^*_{\infty,\lambda}$?
  - How close is $\hat{\phi}^*$ to an optimal stationary deterministic policy?

Approximating Value Functions

Numbering of results & def^{n}s is from Chapter 9 of Chan & Puterman

To develop the framework of approximating value functions, we introduce the notion of basis functions or features.

Definition 9.1 [Basis Function or Feature]

A feature or a basis function is a function defined on $\mathcal{S}$.

$I << |\mathcal{S}|$

We let $\underline{b}(s) = [b_0(s), \ldots, b_I(s)]$ denote the feature vector of state $s \in \mathcal{S}$.
Here,

$b_0, \ldots, b_I : \mathcal{S} \to \mathbb{R}$ are fixed functions known in advance.

If $\mathcal{S} = \{s_1, \ldots, s_M\}$, then we define the feature matrix B as

$$B = \begin{bmatrix} b_0(s_1) & \cdots & b_I(s_1) \\ \vdots & & \\ b_0(s_M) & \cdots & b_I(s_M) \end{bmatrix}_{M\times(I+1)}$$

Usually, B is constructed in such a way that columns of B are linearly independent.

Exercise:

Show that if columns of B are linearly independent, then $B^T B$ is invertible.

Example specifications of features:

(1) If $\mathcal{S} \subseteq \mathbb{R}$, then polynomials, trigonometric functions, exponentials, splines, etc. are used as features.

Eg: $b_0(s) = 1 \quad \forall s$, $b_1(s) = s \quad \forall s$, $b_2(s) = s^2 \quad \forall s$, ...

(2) If $\mathcal{S}$ is a collection of vectors, then features could be individual components of a state, products of components of a state, or any non-linear function of component values.

Eg: Suppose $\mathcal{S} = \{(x_1, \dots, x_5, y_1, y_2) : 0 \le x_i \le 5 \quad \forall i \in \{1, \dots, 5\}, \; 0 \le y_j \le 2 \quad \forall j \in \{1, 2\}\}$

Then, we can define features $b_0, b_1, \dots, b_5, b_{1,1}, b_{1,2}, \dots, b_{5,1}, b_{5,2}, b'_1, b'_2$ as follows:

$$b_0(x_1, \dots, x_5, y_1, y_2) = 1,$$

$$b_i(x_1, \dots, x_5, y_1, y_2) = x_i \quad \text{for } i \in \{1, \dots, 5\}.$$

$$b_{i,j}(x_1, \dots, x_5, y_1, y_2) = x_i x_j \quad \text{for } i \in \{1, \dots, 5\}, \; j \in \{1, 2\}.$$

$$b'_j(x_1, \dots, x_5, y_1, y_2) = y_j \quad \text{for } j \in \{1, 2\}.$$

(3) Checkers:

empty, blue, red, blue king, red king

$$\mathcal{S} = \{(x_1, \dots, x_{32}) : x_i \in \{E, B, R, BK, RK\} \quad \forall i \in \{1, \dots, 32\},$$

$$\underbrace{\sum_{i=1}^{32} 1_{\{B, BK\}}(x_i) \le 12}_{\#\text{ blue pieces}}, \quad \underbrace{\sum_{i=1}^{32} 1_{\{R, RK\}}(x_i) \le 12}_{\#\text{ red pieces}}\}.$$

Possible features are: for all $(x_1, \dots, x_{32}) \in \mathcal{S}$,

- $b_0(x_1, \dots, x_{32}) = 1$

- $b_1(x_1, \dots, x_{32}) = \sum_{i=1}^{32} 1_{\{B\}}(x_i)$,
- $b_2(x_1, \dots, x_{32}) = \sum_{i=1}^{32} 1_{\{BK\}}(x_i)$,
- $b_3(x_1, \dots, x_{32}) = \sum_{i=1}^{32} 1_{\{R\}}(x_i)$,
- $b_4(x_1, \dots, x_{32}) = \sum_{i=1}^{32} 1_{\{RK\}}(x_i)$.

Remarks: (1) The choice of features is dictated by application and subject matter expertise.

(2) If $\mathcal{S} = \{s_1, \dots, s_M\}$, then Choosing

$$b_i(s) = 1_{\{s_i\}}(s), \quad i = 1, \dots, M,$$

is called the tabular representation of the MDP. In this case,

$$B = \begin{bmatrix} b_1(s_1) & \cdots & b_M(s_1) \\ \vdots & & \\ b_M(s_1) & \cdots & b_M(s_M) \end{bmatrix} = I_{M \times M}.$$

Feature-Based Value Function Approximators

We will now study some feature-based value function approximators (a.k.a. architectures).

Linear Architectures:

A linear architecture / approximator refers to value approximation of the form

$$\underline{V}(s) \approx \beta_0 b_0(s) + \beta_1 b_1(s) + \cdots + \beta_I b_I(s), \quad s \in \mathcal{S},$$

where $b_0(s) = 1$ for all $s \in \mathcal{S}$. In vector notation,

$$\underline{V} \approx B\underline{\beta}, \text{ where}$$

$$B = \begin{bmatrix} b_0(s_1) & \cdots & b_I(s_1) \\ \vdots & & \\ b_0(s_m) & \cdots & b_I(s_m) \end{bmatrix}_{m \times (I+1)}, \quad \underline{\beta} = \begin{bmatrix} \beta_0 \\ \vdots \\ \beta_I \end{bmatrix}_{(I+1) \times 1}.$$

Non-linear architectures

A non-linear architecture / approximator is a value approximation using the form

$$\underline{V}(s) \approx f_{\underline{\beta}}\left(b_0(s), \ldots, b_I(s)\right), \quad s \in \mathcal{S},$$

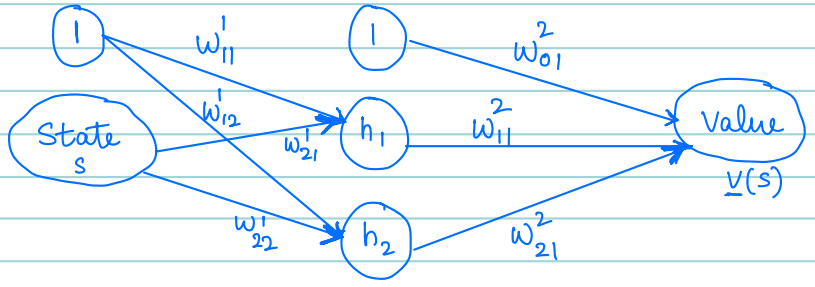
where $\underline{\beta}$ is a vector of parameters that parametrises the approximating function.

Eg: $f_{\underline{\beta}}\left(b_0(s), b_1(s)\right) = \beta_0 b_0(s) + \beta_1 e^{\beta_2 b_1(s)}, \quad s \in \mathcal{S}.$

Here,

$$B = \begin{bmatrix} b_0(s_1) & b_1(s_1) \\ \vdots & \vdots \\ b_0(s_m) & b_1(s_m) \end{bmatrix}_{m \times 2}, \quad \underline{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}_{3 \times 1}.$$

Neural Networks



Assuming the sigmoid activation function $\sigma(x) = \frac{1}{1+e^{-x}}$,

we have:

Input to h_1: $w^1_{11} + w^1_{21} s$,

Input to h_2: $w^1_{12} + w^1_{22} s$.

$$V(s) = w^2_{01} + w^2_{11}\,\sigma\left(w^1_{11} + w^1_{21} s\right) + w^2_{21}\,\sigma\left(w^1_{12} + w^1_{22} s\right)$$

$$= w^2_{01} + w^2_{11} \frac{1}{1+e^{-(w^1_{11} + w^1_{21} s)}} + w^2_{21} \cdot \frac{1}{1+e^{-(w^1_{12} + w^1_{22} s)}}$$

Parameter Estimation for Value Function Approximation: Linear Architectures

We saw that a linear architecture approximates value as

$$v(s) \approx \beta_0 + \beta_1 b_1(s) + \cdots + \beta_I b_I(s), \quad s \in \mathcal{S}.$$

How do we determine $\beta_0, \cdots, \beta_I$ that best approximate $v(\cdot)$?

Thought:

Approximate the underlying operator instead of $\underline{v}(s)$ for $s \in \mathcal{S}$.